

AhamX Bodhi

Where I Becomes Infinite

What is a Transformer?

Module 2.2: Transformer Architecture Fundamentals
Prof. Sashikumaar Ganesan · IISc Bengaluru · 2026



What We Will Cover

Concept Roadmap

- The problem that Transformers were designed to solve
- Architecture overview: the big picture before the details
- Encoder and Decoder — what they do and when each is used
- The core innovation: self-attention in one sentence
- Three major Transformer families and their applications
- Where Transformers fit in the broader AI landscape
- Common misconceptions and knowledge connections



Learning Objectives

By the End of This Concept You Will Be Able To

- **Define** a Transformer and state the central innovation that makes it different from prior sequence models (RNNs, LSTMs)
- **Identify** the high-level components: input embedding, positional encoding, encoder stack, decoder stack, and output projection
- **Distinguish** encoder-only, decoder-only, and encoder-decoder architectures and name a representative model for each family
- **Explain** in plain language why self-attention allows parallel processing of sequences
- **Connect** the Transformer to downstream concepts: self-attention, multi-head attention, positional encoding, and fine-tuning





The Sequence Modelling Challenge

What Came Before: RNNs and LSTMs

- Recurrent Neural Networks (RNNs) process tokens one at a time, left to right
- Each step depends on the hidden state from the previous step — **no parallelism**
- Long-range dependencies are hard: information from early tokens gets diluted
- LSTMs improved memory retention but still processed tokens **sequentially**
- Training time scales poorly with sequence length on modern GPU hardware

The fundamental bottleneck: **sequential computation** prevented effective use of parallel hardware and made long-context modelling unreliable.



Attention Is All You Need (2017)

The Paper That Changed Everything

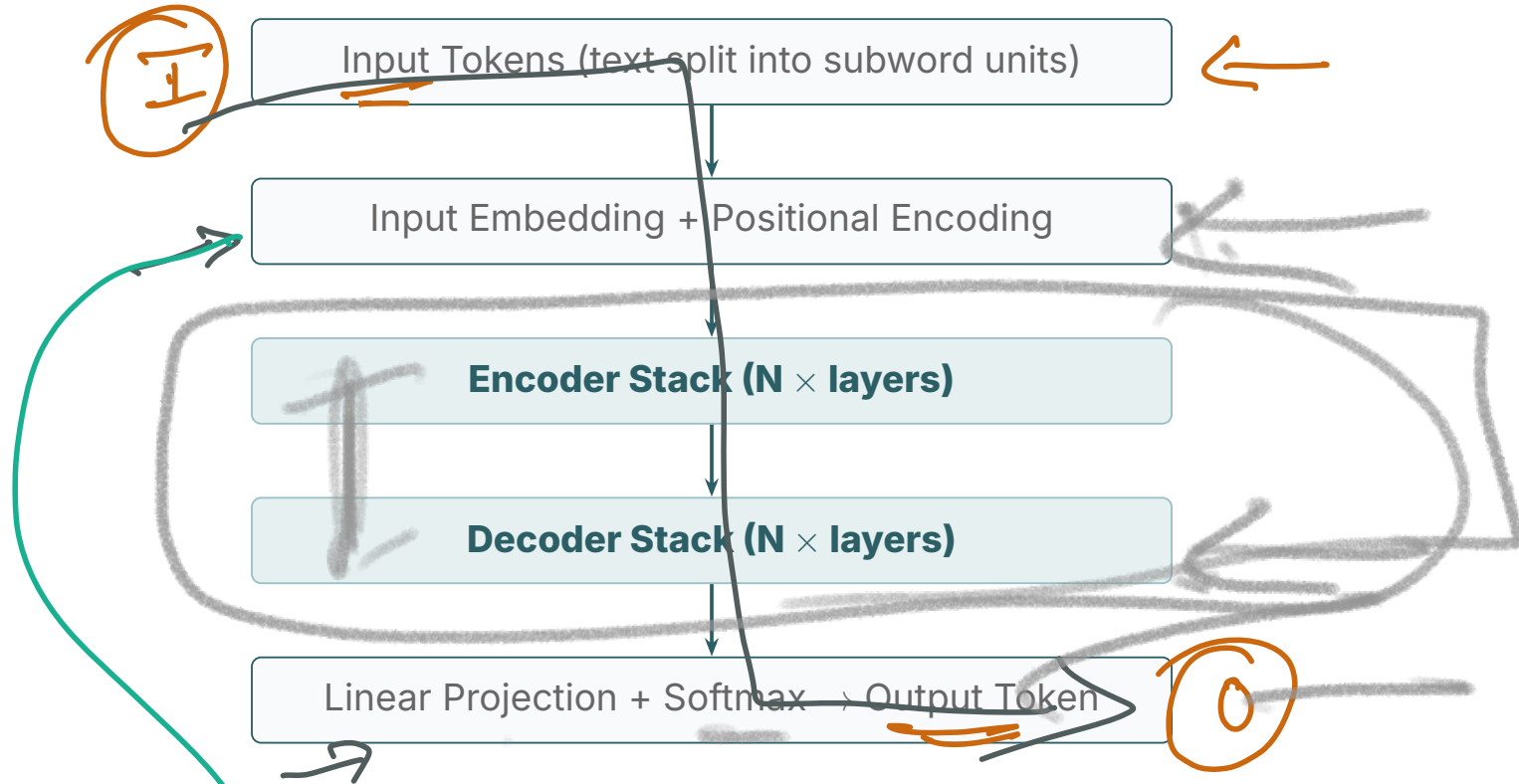
- Vaswani et al., Google Brain, NeurIPS 2017
- Proposed replacing recurrence entirely with **self-attention**
- All tokens attend to all other tokens **simultaneously** — full parallelism
- Trained faster and achieved better results on machine translation benchmarks
- Became the foundation architecture for GPT, BERT, T5, Claude, Gemini, and LLaMA

The key insight: **attention**, not recurrence, is the right primitive for modelling relationships in sequences.





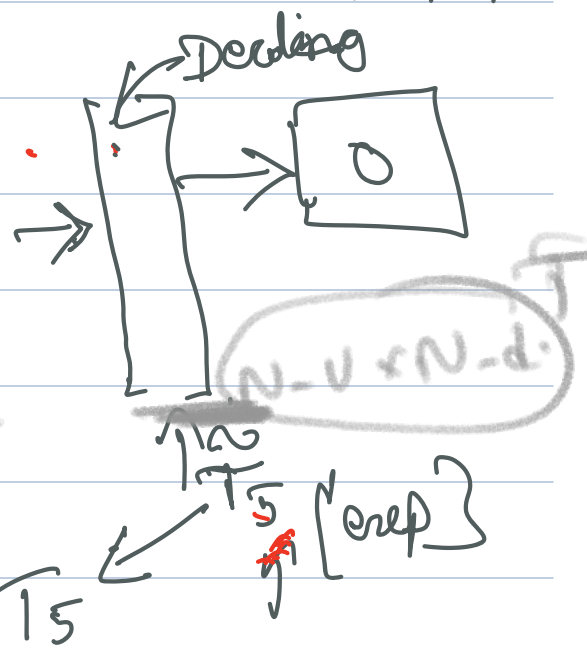
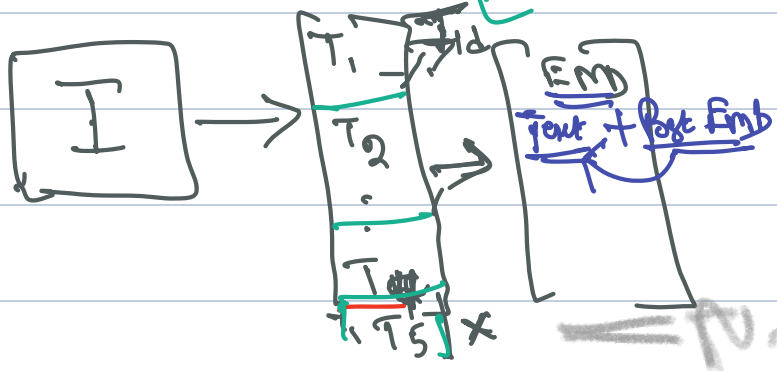
The Transformer --- High-Level View



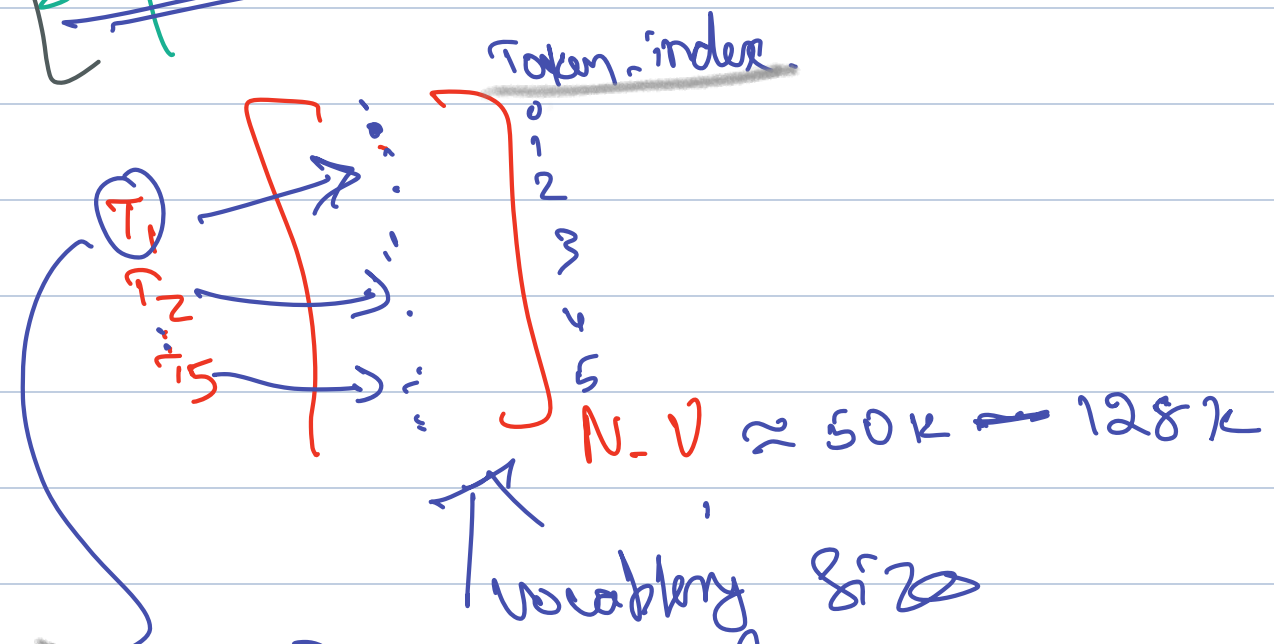
Note: Decoder-only models (GPT, Claude, LLaMA) omit the Encoder. Encoder-only models (BERT) omit the Decoder.

I like Cat very much

T_1 T_2 T_3 T_4 T_5



Sequence



$n-d$ or d -model $\leftarrow$ hidden size / vector / word dim.

$T_5 = \begin{bmatrix} \vdots \\ \vdots \end{bmatrix}$



Before the Encoder: Embedding and Positional Encoding

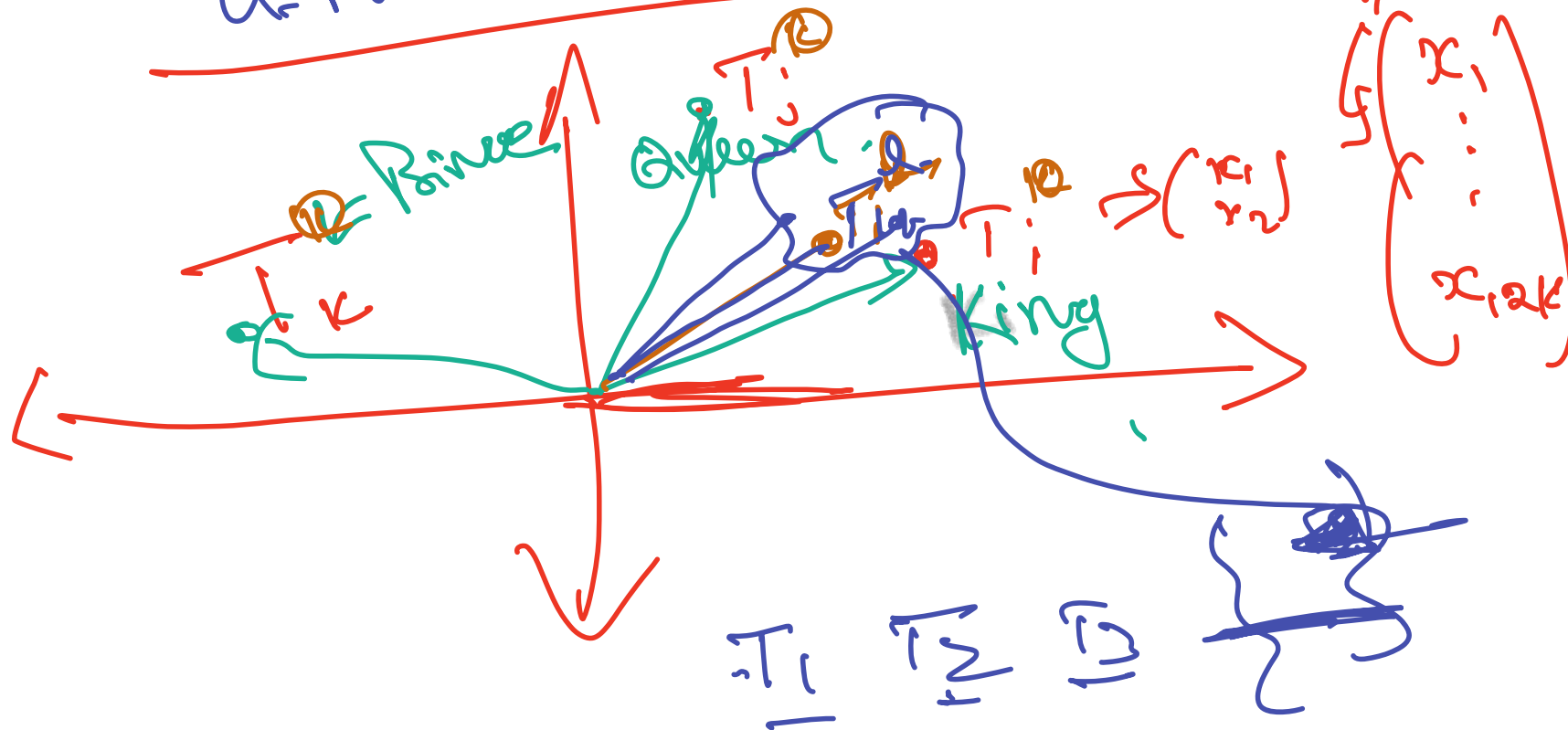
Input Embedding

- Each token is mapped to a dense vector of dimension d_{model} (e.g., 512 or 768)
- The embedding matrix is **learned** during training
- Tokens with similar meanings cluster in embedding space
- Output: a matrix of shape $\text{seq_len} \times d_{\text{model}}$

Positional Encoding

- Attention has **no inherent order** — it sees a set, not a sequence
- Positional encodings inject the position of each token into its vector
- Original paper used **sinusoidal functions**; modern models use learned or rotary encodings (RoPE)
- Added element-wise to the embedding vector

$d_{\text{model}} \approx 12K$





Self-Attention in One Sentence

The Central Idea

- For each token in the sequence, self-attention computes a **weighted sum** of all other tokens' representations, where the weights reflect how **relevant** each token is to the current one
- The word "bank" in "river bank" attends more to "river" than to "money"
- This relevance is learned from data — not hand-coded
- It runs **in parallel** across all positions simultaneously

Formally: each token produces three vectors — **Query (Q)**, **Key (K)**, and **Value (V)**. The attention score between two tokens is $\text{score}(Q_i, K_j) = Q_i \cdot K_j / \sqrt{d_k}$, then softmax-normalised.

$$\text{Softmax} \left(\frac{Q_i K_j}{\sqrt{d_k}} \right)$$

w^a

$\begin{bmatrix} \end{bmatrix}_{n_a \times n_d}$

w_i^k

n_k

$w_i^a =$

w_i^a

$\begin{bmatrix} \end{bmatrix}_{n_a \times 1}$

w^a

$\begin{bmatrix} \end{bmatrix}_{n_a \times n_d}$

$\begin{bmatrix} T_i \end{bmatrix}_{n_d \times 1}$

$i=1, \dots, 5$

w_i^k

w_1^a

w_2^a

w_3^a

w_4^a

w_5^a

w_1^k

w_2^k

w_3^k

w_4^k

w_5^k



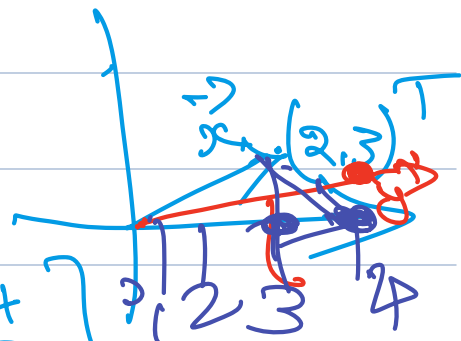
$$w_1^a \cdot w_2^k = \sum_{i=1}^{n_a n_k} w_{2,i}^a w_{1,i}^k$$

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

W 142

$$A \cdot \vec{x} = \vec{y}$$

$$\vec{y} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \quad \vec{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad \vec{y} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$



$$A = \begin{bmatrix} 2 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = 4$$

$$A = \begin{bmatrix} 2 & 0 \\ 3 & 1 \\ 0 & 5 \\ 5 & 6 \end{bmatrix} \quad \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} \end{bmatrix}$$

6×2
 2×1
 4×1



Query, Key, Value --- The Attention Mechanism

Query Q_i

- "What am I looking for?"
- Produced by projecting token x_i through learned weight matrix W^Q
- One query vector per token position

Key K_j

- "What do I contain?"
- Produced by projecting token x_j through learned weight matrix W^K
- Dot product $Q_i \cdot K_j$ gives a raw relevance score

Value V_j

- "What do I contribute?"
- Produced by projecting token x_j through learned weight matrix W^V
- The attention output is a weighted sum of all V_j vectors

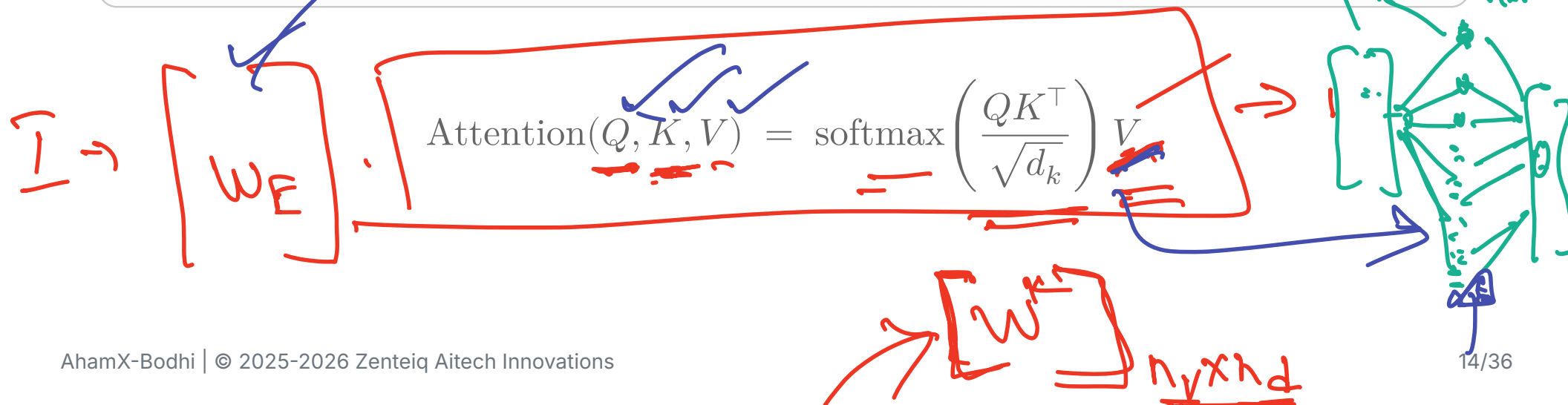
Attention $\approx \frac{1}{3} \left\{ \begin{matrix} \text{Total} \\ \text{Query} \end{matrix} \right\}$

The Attention Formula



Putting Q, K, V Together

- Compute a score for every (i, j) pair: $Q_i \cdot K_j$, then scale by $1/\sqrt{d_k}$
- Apply softmax across all j to get attention weights α_{ij} (they sum to 1)
- The output for position i is the weighted sum $\sum_j \alpha_{ij} V_j$
- Because all positions are processed simultaneously, there is **no sequential dependency**

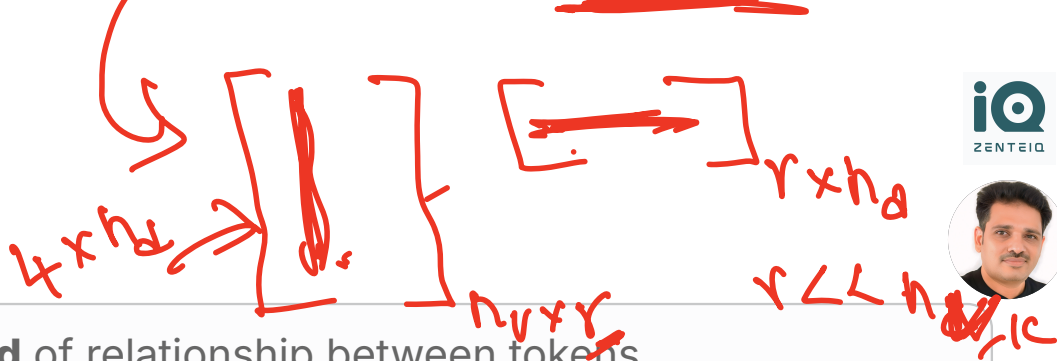


Multi-Head Attention



Why Multiple Attention Heads?

- A single attention head learns **one kind** of relationship between tokens
- Multi-head attention runs h attention operations in parallel, each with its own learned Q, K, V projections
- Different heads specialise: one might track syntactic dependencies, another coreference, another positional proximity
- Outputs from all heads are concatenated and linearly projected back to d_{model}
- The original "Attention Is All You Need" paper used $h = 8$ heads



Multi-head attention is the main computational block in both the encoder and the decoder. It is the concept that immediately follows this one in Module 2.2.

→ Kv - Cash / Flash Attor

→ MFA 240

W_E W'_Q W'_{12} W_V W_N

W_Q^2 W_{12}^2

$\begin{pmatrix} 0 & \dots & x_i & \dots \\ \vdots & & & \end{pmatrix}$

$\begin{bmatrix} -702 \\ 80 \\ \vdots \\ 1200 \\ \vdots \\ -100 \end{bmatrix}$ $\rightarrow$ $\begin{pmatrix} x_i \\ 0 \\ 200 \end{pmatrix}$



The Encoder Block

What the Encoder Does

- Receives the full input sequence and produces a **rich contextual representation** of each token
- Each encoder layer contains two sub-layers:
 - **Multi-Head Self-Attention** — each token attends to all other tokens
 - **Feed-Forward Network (FFN)** — applied independently to each position
- Each sub-layer is wrapped with a **residual connection** and **Layer Normalisation**
- The encoder processes the **entire input in parallel** — no sequential step
- Stacking N encoder layers creates progressively more abstract representations



The Decoder Block

What the Decoder Does

- Generates output tokens **one at a time**, autoregressively
- Each decoder layer contains three sub-layers:
 - **Masked Self-Attention** — token can only attend to previous tokens (causal masking prevents “seeing the future”)
 - **Cross-Attention** — each decoder token attends to the encoder output
 - **Feed-Forward Network (FFN)** — same as in the encoder
- Causal masking ensures the model cannot use future tokens during training or generation
- Cross-attention is what allows translation: the decoder “reads” the source representation while generating the target



Encoder vs Decoder --- At a Glance

Property	Encoder	Decoder
Attention type	Bidirectional self-attention	Causal (masked) self-attention
Token visibility	All tokens see each other	Each token sees only past tokens
Output type	Contextual embeddings	Next-token probability distribution
Parallelism	Fully parallel	Sequential at inference
Typical use	Classification, NER, QA	Text generation, chat, completion
Example model	BERT, RoBERTa	GPT-4, Claude, LLaMA





Three Transformer Families

Encoder-Only

- Bidirectional context
- Best for: classification, NER, semantic search
- Models: BERT, RoBERTa, DistilBERT
- Input → representation (no generation)

Decoder-Only

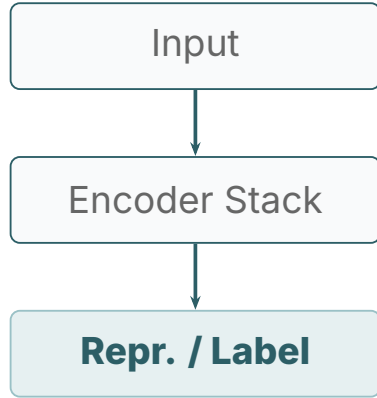
- Causal / left-to-right context
- Best for: text generation, chat, code
- Models: GPT-4, Claude, LLaMA, Mistral
- The dominant family for modern LLMs

Encoder-Decoder

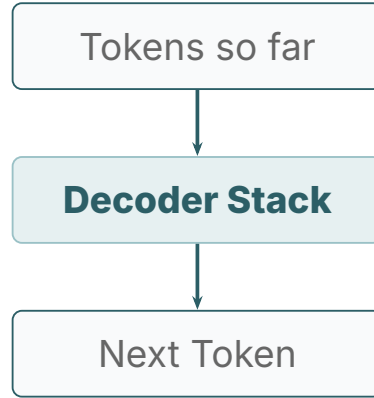
- Encoder reads input; decoder generates output
- Best for: translation, summarisation, seq2seq
- Models: T5, BART, mBART, MarianMT
- Cross-attention links encoder to decoder



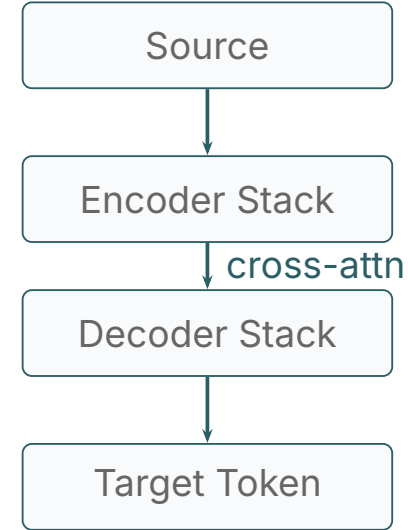
Architecture Families --- Visual Comparison



**Encoder-Only
(BERT)**



**Decoder-Only
(GPT, Claude)**



**Enc-Dec
(T5, BART)**





Feed-Forward Network and Residual Connections

Position-Wise FFN

- Applied **independently** to each token position
- Two linear layers with a non-linearity (ReLU or GELU) in between
- Inner dimension d_{ff} is typically $4 \times d_{\text{model}}$ (e.g., 2048 for $d_{\text{model}} = 512$)
- Provides additional capacity beyond attention
- Can be thought of as a “memory” of learned facts

Residual Connections and LayerNorm

- Each sub-layer is wrapped: **output = $\text{LayerNorm}(x + \text{sub-layer}(x))$**
- Residual connections prevent vanishing gradients in deep stacks
- Layer Normalisation stabilises activations at each layer
- These two techniques are what make training dozens of layers practical
- Standard GPT-2 had 48 layers; GPT-4 is estimated at 96–120 layers





Transformers in Production --- Text and Code

Real-World Applications

- **Conversational AI (ChatGPT, Claude, Gemini):** Decoder-only Transformers generate human-quality dialogue by predicting the next token conditioned on the full conversation history
- **Code Generation (GitHub Copilot, Cursor):** Same architecture applied to code tokens; the FFN layers store substantial “knowledge” of programming idioms and APIs learned from open-source repositories
- **Document Summarisation (Notion AI, Jasper):** Long-context Transformers (GPT-4-128k, Claude 3) ingest full documents in the context window and generate abstractive summaries token by token
- **Enterprise Search (Elastic with BERT):** Encoder-only models produce dense vector embeddings for semantic similarity search, replacing keyword matching



Transformers in Production --- Translation and Multimodal

More Production Systems

- **Machine Translation (DeepL, Google Translate):** Encoder-Decoder Transformers (mBART, MarianMT) encode the source sentence and decode the target language; the cross-attention mechanism is what enables meaning transfer
- **Retrieval-Augmented Generation (Perplexity AI):** A Transformer retriever (dense encoder) finds relevant passages; a decoder-only LLM generates the answer conditioned on retrieved context
- **Vision-Language Models (GPT-4V, Gemini Ultra):** Image patches are treated as tokens and fed into the Transformer alongside text; the same self-attention mechanism integrates visual and linguistic context
- **Protein Structure Prediction (AlphaFold 2):** Amino acid sequences are processed by a Transformer variant; attention reveals which residues interact at distance — a purely sequence-to-structure task



Why Transformers Scale

The Scaling Properties of the Architecture

- **Parallelism:** Self-attention processes all tokens simultaneously — ideal for modern GPU/TPU matrix-multiply hardware (cuBLAS, XLA)
- **Depth:** Residual connections allow stacking hundreds of layers; each layer adds representational capacity without exploding gradients
- **Width:** Increasing d_{model} and d_{ff} adds parameters smoothly; Scaling Laws (Kaplan et al., 2020) quantify the return
- **Data:** Self-supervised training on raw text needs no labels — the entire internet is training data
- **Transfer:** A pre-trained Transformer is a universal foundation; fine-tuning it costs orders of magnitude less than training from scratch

These four properties together explain why **every frontier LLM** today is Transformer-based.





Common Misconceptions

Do Not Confuse These

- **"Transformer" = ChatGPT:** The Transformer is an *architecture*. ChatGPT is a *product* built on a fine-tuned decoder-only Transformer (GPT-4). BERT, T5, and Claude are all Transformers but are quite different.
- **"Self-attention looks up a dictionary":** Attention scores are *computed* from learned projections, not retrieved from a fixed lookup table. They change with every input.
- **"Transformers process sequences left-to-right":** The encoder is fully bidirectional. Only *decoder* generation is left-to-right due to causal masking.
- **"The FFN is unimportant compared to attention":** The FFN accounts for $\sim 2/3$ of model parameters and stores factual knowledge. Ablation studies show its removal causes severe performance degradation.



Prerequisites and Where You Are

You Should Already Know

- What a Language Model is (Module 1.2)
- Autoregressive generation and next-token prediction
- Foundation Models Overview
- Basic linear algebra: matrix multiplication, vectors

Your Position in Module 2.2

- This concept is the **entry point** to the Transformer Architecture track
- It provides the vocabulary needed for all deeper dives that follow
- Completing it unlocks self-attention, multi-head attention, positional encoding, and the full training pipeline



What This Concept Unlocks --- Part 1

Immediate Next Concepts in Module 2.2

- **Self-Attention Mechanism** (AI-TR-FN-TH-000002) — the mathematical derivation of Q , K , V , and the attention score formula
- **Multi-Head Attention** (AI-TR-FN-TH-000003) — how and why multiple heads are used in parallel
- **Positional Encoding** (AI-TR-FN-TH-000004) — sinusoidal and rotary approaches to injecting sequence order
- **Embeddings and Token Representations** (AI-TR-FN-TH-000005) — how token and positional embeddings are learned and used



What This Concept Unlocks --- Part 2

Downstream Concepts Across the Programme

- **Tokenization** (Module 2.3) — the preprocessing step that feeds token IDs into the embedding layer
- **LLM APIs and Parameters** (Module 3) — temperature and top- p operate on the Transformer's output softmax distribution
- **Fine-Tuning** (Module 7) — updating the Transformer's learned weights on domain-specific data
- **Pre-Training from Scratch** (Module 9) — building and training the full Transformer stack on a raw corpus
- **LoRA / QLoRA** (Module 7) — parameter-efficient methods that add low-rank adapters to Transformer weight matrices



Related Concepts Across the Programme

Cross-Module Connections

- **RLHF** (Module 7) — the reward model and the policy model are both Transformers; the architecture is unchanged, only the training signal differs
- **RAG Architecture** (Module 5) — the retriever and the reader are both Transformer-based; the architecture is the enabling technology
- **Agentic AI** (Module 6) — LLM agents use a Transformer as the “brain”; tool calls are structured tokens in the sequence
- **Quantization and Compression** (Module 9) — techniques like 4-bit quantization and knowledge distillation act directly on Transformer weights
- **Model Sizing and Scaling Laws** (Module 1.2) — the parameters counted in scaling laws are the weights in Transformer layers



Key Takeaways

Remember These Five Points

- The Transformer replaced sequential RNNs with **parallel self-attention**, enabling training on internet-scale data
- Each Transformer layer alternates **multi-head self-attention** and a **position-wise feed-forward network**, wrapped with residual connections and layer normalisation
- Three families serve distinct purposes: **encoder-only** (classification), **decoder-only** (generation), and **encoder-decoder** (translation/seq2seq)
- Transformers scale gracefully: more parameters, more data, and more compute consistently improve performance as described by the Scaling Laws
- Virtually every frontier AI system — GPT-4, Claude, Gemini, LLaMA, BERT, T5, AlphaFold — is Transformer-based; understanding this architecture is the single highest-leverage investment in this programme

Thank You

What is a Transformer? — Module 2.2
Next: Self-Attention Mechanism